`yield from` and Closing Generators

`yield from` delegates to a sub-iterator; `close()` and `finally` guarantee clean-up.

Delegate with yield from

```
def inner():
    yield 1
    yield 2

def outer():
    yield 0
    # re-yield everything from inner
    yield from inner()
    yield 3

print(list(outer()))
```

OUTPUT

```
[0, 1, 2, 3]
```

`yield from it` replaces `for x in it: yield x` – and also forwards `send()`, `throw()` and the sub-generator's `return` value.

Flatten nested lists recursively

```
def flatten(items):
    for item in items:
        if isinstance(item, list):
            yield from flatten(item)    # recurse
        else:
            yield item

print(list(flatten([1, [2, [3, 4]], 5])))
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

Capture the sub-generator's return value

```
def sub():
    yield "a"
    return "sub-result"

def main():
    # result = sub's return value
    result = yield from sub()
    print("sub returned", result)
    yield "b"

print(list(main()))
```

OUTPUT

```
sub returned sub-result
['a', 'b']
```

Ways a generator ends

How	What happens
runs out of items	`StopIteration`
`return v`	`StopIteration(v)` – value in `.value`
`g.close()`	`GeneratorExit` at the `yield`; `finally` runs
exception escapes	propagates to the caller
garbage-collected	Python calls `close()` for you

close() runs your finally block

```
def reader():
    print("open")
    try:
        for i in range(10):
            yield i
    finally:
        print("close")       # always runs

r = reader()
print(next(r), next(r))
# GeneratorExit raised inside
r.close()
print("closed")
```

OUTPUT

```
open
0 1
close
closed
```

Never yield while closing

```
def bad():
    try:
        yield 1
    except GeneratorExit:
        yield "nope"         # not allowed!

g = bad(); next(g)
try:
    g.close()
except RuntimeError as e:
    print(e)
```

OUTPUT

```
generator ignored GeneratorExit
```

Deterministic clean-up with closing()

```
from contextlib import closing
with closing(reader()) as r:
    print(next(r))
```

OUTPUT

```
open
0
close
```

TIP Generators holding files, locks or connections should use `try/finally`; callers use `for`, `with closing(...)` or `.close()` so clean-up is never left to chance.